An Energy-Aware Extension of Raft with Observer Roles and Provisional Consensus for IoT Clusters

Ashwanth Kumaravel Sreenandu G
Vellore Institute of Technology, Chennai, India
Email: ashwanth.kumaravel2023@vitstudent.ac.in, sreenandu.g2023@vitstudent.ac.in

Abstract—Raft [1] is widely used for replication in distributed systems, but its assumptions of homogeneous, always-powered participants and periodic heartbeats are expensive in battery-powered IoT clusters. Recent work has extended Raft along three broad axes: energy-aware operation (EDRaft [10], Energy-Efficient Raft [9], Weighted RAFT [11]), lightweight or witness-style participants [8], [17], and hierarchical or adaptive leader selection (LH-Raft [13], MCRaft [14], DQN-Raft [15], LRD-Raft [16]). We present Echo, a crash-fault-tolerant variant of Raft that unifies three of these ideas into a single protocol: (i) an observer role for coordinators whose battery drops below a configurable threshold, with hysteretic re-entry; (ii) an event-driven consensus trigger mechanism that replaces periodic full AppendEntries heartbeats with ∼32-byte liveness pings plus on-demand replication rounds; and (iii) provisional consensus, a partition-tolerance mechanism that lets sub-clusters make local progress under a distinct partition epoch and reconciles on heal while preserving Raft's safety properties on the globally-committed prefix. Measuring Echo against Raft under an identical, matched sensor workload–fixing an asymmetry present in an earlier version of this harness–exposed Echo's own inefficiencies: a liveness ping broadcast to every node accounts for 75–80% of its total traffic, and unweighted random election timeouts repeatedly split votes. We therefore present a second protocol, echoD, a Raft/Echo hybrid that keeps Echo's tiered topology and provisional consensus but adds six targeted optimizations–edge-side delta filtering, batched event-driven consensus, coordinators-only adaptive liveness, battery-ordered election timeouts, directed leader handoff, and batched partition reconciliation–none of which weakens Raft's safety argument. On a matched 5-coordinator, 10-leaf, 5-second workload, echoD uses 4–6× fewer messages than Raft and Echo while keeping commit latency and cluster availability equal or better. We implement all three protocols on a shared asyncio message bus (≈4 KLOC Python) and release a hardware-free MQTT system that runs any of the three over real network transport, with a Flask+Socket.IO dashboard. The codebase is open source.

Index Terms—distributed consensus, Raft, Internet of Things, energy-aware protocols, network partitions, MQTT, edge computing.

I. Introduction

The steady growth of Internet-of-Things (IoT) deployments has intensified the need for consensus algorithms that are robust to the particular constraints of edge environments: limited device power, intermittent wireless connectivity, and heterogeneous node capabilities. Raft [1], [2] has become the default consensus building block for many IoT and blockchain systems because it is simpler to understand and implement than Paxos [3] while providing equivalent safety guarantees.

Standard Raft is not, however, optimised for energy efficiency or for the dynamic role adaptation that IoT clusters demand [7], [8], [10]. Three concrete pain points recur:

1) Low-battery nodes remain full voters until they fail. A coordinator near depletion can still be elected leader and then die mid-term, forcing re-elections that further drain healthy peers [9], [10].
2) Idle traffic is dominated by heartbeats. Empty AppendEntries RPCs every 50–150 ms dominate per-node transmit energy in duty-cycled deployments, even when nothing needs to be replicated [8], [11].
3) Strict partition liveness is undesirable. Many IoT applications–local telemetry, rate-limited actuation–prefer bounded local progress with deferred reconciliation over blocking all writes until the network heals [16], [17].

Contributions: This paper presents Echo, an extension of Raft targeting energy-constrained clusters. Concretely:

- A battery-aware observer role with hysteretic re-entry (T_{low}=15%, T_{restore}=25%), an energy-gated vote predicate, and an energy-weighted election tiebreaker. This draws on the witness/observer idea of Pâris and Long [8] and the layered node classification of PyCloudIoT [17], but integrates it into the Raft election path directly rather than as a separate tier.
- Event-driven replication triggers. Full AppendEntries traffic is replaced with ∼32-byte liveness pings, and real replication only fires on typed triggers (Delta, Join, Partition, Reconcile, Liveness). This is a natural generalisation of the message-reduction goals of EDRaft [10] and Energy-Efficient Raft [9].
- Provisional consensus. A partition-epoch field in AppendEntries allows sub-clusters to replicate locally under partition; a deterministic reconciliation procedure truncates losing-side provisional entries and reproposes them as new commands on heal, preserving the Raft safety argument on the reconciled committed prefix.
- echoD, a Raft/Echo hybrid that keeps Echo's tiered topology, energy gating, and provisional consen-

sus, and adds six efficiency optimizations–edge-side delta filtering, batched event-driven consensus, coordinators-only adaptive liveness, battery-ordered election timeouts, directed leader handoff, and batched partition reconciliation–motivated directly by measuring where Echo's own traffic was wasted.
- A shared implementation of Raft, Echo, and echoD over a common asyncio message bus for like-for-like measurement, plus a hardware-free MQTT system that runs any of the three protocols over real network transport, with a Flask+Socket.IO dashboard.
- An empirical methodology with an identical, seeded workload replayed across all three protocols, plus an experiment harness that reproduces the same metrics over real MQTT transport for future hardware validation.

The paper is organised as follows. Section II reviews Raft and summarises the literature on energy-aware, witness-based, hierarchical, and reinforcement-learning-based Raft variants. Section III details the Echo protocol. Section IV presents echoD, the six optimizations it adds on top of Echo, and its positioning relative to prior art. Section V describes the implementation. Section VI sketches the safety argument. Section VII presents the evaluation methodology and matched-workload results for all three protocols. Sections VIII–IX discuss limitations, open questions, and next steps.

II. Background and Related Work

A. Raft in one paragraph

Raft [1] elects a single leader per monotonically increasing term using randomised election timeouts. The leader replicates log entries via AppendEntries RPCs; an entry is committed once a majority of servers has persisted it. Safety rests on five properties–election safety, leader append-only, log matching, leader completeness, and state-machine safety. Howard's ARC thesis [7] gives a formal treatment of these properties and is the reference used throughout this paper.

B. Energy-aware Raft variants

EDRaft [10] reframes leader selection as a collective decision that prioritises group uptime over individual optimality; the authors report leader uptime up to 70% longer than standard Raft under extended low-power runs. Nakagawa and Hayashibara's Energy-Efficient Raft [9] reduces the message count per round by coalescing control traffic and duty-cycling non-leader nodes. Weighted RAFT [11] assigns per-node weights that reflect compute or network cost and uses them to bias the election, reducing unnecessary leader churn in heterogeneous IoT blockchains. Echo's contribution here is the observer role: rather than weighting votes, low-energy nodes leave the voting set entirely, and re-enter only after a hysteresis gap.

C. Witness and lightweight-participant roles

Pâris and Long [8] were among the first to show that replacing some full replicas with witnesses–nodes that vote but do not store state–reduces the energy footprint of a distributed consensus cluster while preserving fault tolerance. PyCloudIoT [17] extends this to an energy-efficient FaaS edge platform that classifies nodes by energetic and computational capability and orchestrates them accordingly. Our observer role is compatible with both of these ideas but is dynamic: a coordinator that begins as a full voter can become an observer mid-deployment when its battery drops, and can rejoin later.

D. Hierarchical and adaptive models

LH-Raft [13] forms location- or reputation-based candidate groups so that leader elections are localised before global agreement is reached; this reduces communication overhead in large IoT-blockchain deployments. MCRaft [14] distributes leadership among multiple super-nodes to improve cluster stability. Zuo et al. [20] group Raft peers using affinity-propagation clustering to reduce inter-group traffic. Our tiered topology (coordinators + leaves) is compatible with these designs: leaves could be clustered under a subset of coordinators along LH-Raft lines, which we leave as future work.

E. Reinforcement-learning-based leader selection

DQN-Raft [15] and its successor DQN-Raft+ [19] use deep Q-learning to choose leaders based on observed reward signals, improving adaptability on heterogeneous networks. LRD-Raft [16] decouples log replication from leader traffic to balance load and improve energy efficiency in consortium-blockchain IoT. These techniques are orthogonal to Echo's contributions; a "DQN-Echo" that learns when to lower T_{low} dynamically is a natural future direction. Hybrid Raft–PoW designs [18] take yet another angle by composing Raft with a proof-of-work layer for tamper resistance.

F. Partition-tolerant consensus

Classical Raft and systems built on it tolerate partitions by blocking progress on the minority side. Eventual-consistency stores allow partition writes at the cost of weaker semantics. Variants such as Flexible Paxos [4] and EPaxos [5] tune quorum shape for latency or locality but do not introduce explicit provisional writes. Echo occupies the intermediate design point: provisional entries are never globally committed, so reads over the committed prefix remain linearisable, but each sub-cluster can durably accumulate locally ordered proposals that are re-proposed after heal.

G. Energy-aware routing at the same layer

Outside the consensus literature, a large body of work on energy-aware routing in IoT and wireless sensor networks informs the constants used in this paper: Shah et

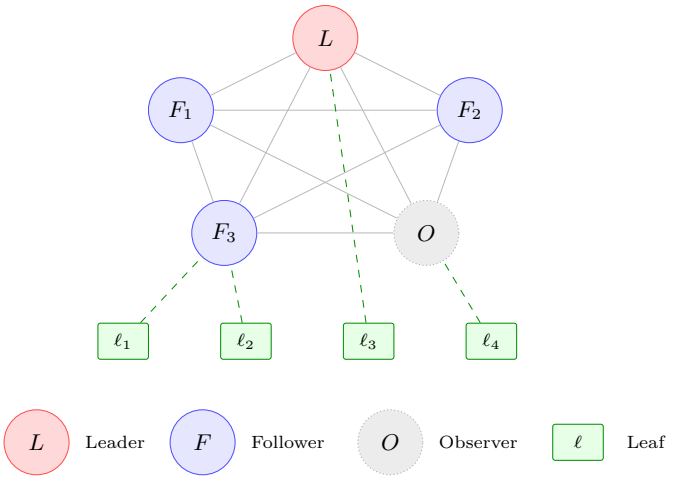

Fig. 1. Tiered Echo cluster. Coordinators (L, F_i, O) form a full mesh and run consensus; leaves ℓ_i register with a coordinator and only send sensor reports. An observer is a coordinator whose battery has dropped below T_{low}; it has left the voting quorum but still receives log updates.

al.'s throughput-optimised clustered routing [21], Samadi et al.'s SDN-based intelligent energy-aware routing [22], and Kang and Jeon's UAV-enabled aggregation [23]. These provide useful priors for realistic drain rates; they do not change the protocol but they do constrain the experiment design.

III. ECHO Protocol Design

A. Topology and node roles

An Echo cluster is organised in two tiers (Fig. 1):

- Coordinators run the consensus state machine and replicate a log. A typical deployment uses 3–7 coordinators.
- Leaves are lightweight sensor sources. They register with a coordinator and forward SensorDataReport messages; they do not participate in voting.

B. States and energy gating

A coordinator's state space extends Raft with two additional states (Fig. 2):

- Observer–entered from any non-observer state when battery $< T_{\text{low}}$ (default 15%). An observer does not vote, is not counted toward the active quorum, and cannot transition to candidate. It still receives AppendEntries and keeps its log current. It re-enters as Follower when battery $\geq T_{\text{restore}}$ (default 25%). The hysteresis gap prevents oscillation.
- Local leader–a coordinator that was leader when a partition was detected. It continues to replicate within its sub-cluster under a non-zero partition epoch and rejoins as follower after reconciliation.

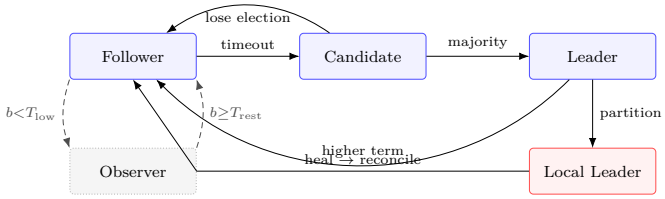

Fig. 2. Coordinator state machine. Solid arrows: standard Raft. Dashed arrows: Echo's energy gate (also applies from Candidate/Leader, not drawn for clarity). Leader transitions to Local Leader on partition and reconciles back to Follower on heal.

C. Energy-aware voting

Every RequestVote RPC carries a battery_level field in $[0, 100]$. A voter rejects the RPC if any standard Raft check fails or if battery_level $< T_{\text{low}}$. In effect, low-energy nodes cannot become leaders, and the cluster does not pay the cost of re-electing after their foreseeable depletion.

For tie-breaking among simultaneous candidates, we use an energy-weighted election score:

$$\text{score}(C) \;=\; |\text{votes}(C)| \cdot \frac{\text{battery}(C)}{\max_{C'} \text{battery}(C')}. \qquad (1)$$

The simple-majority rule still decides correctness; (1) affects tiebreaks only. This is an important detail for the safety argument (§VI): no predicate is added to the committed path.

D. Event-driven consensus triggers

In classical Raft the leader emits a full AppendEntries RPC every heartbeat interval, regardless of whether there is anything to replicate. Echo replaces this with two mechanisms:

1) Liveness pings (≈32 bytes on the wire) carry only (term, leader_id, timestamp) and solely reset followers' election timers.
2) Trigger-driven AppendEntries only fires when a typed trigger is raised: Delta (sensor reading changes by more than Δ_{thr}=5%), Join (leaf registration), Partition, Reconcile, or Liveness. Followers buffer triggers that arrive while no leader is known; a newly-elected leader flushes the buffer immediately on promotion.

E. Provisional consensus and reconciliation

Every AppendEntriesRPC carries a partition_epoch integer (0 in normal operation, incremented by enter_provisional_mode() on partition). Log entries appended under a non-zero epoch are tagged accordingly.

Fig. 3 shows the full flow. When a coordinator observes a non-zero partition_epoch in an incoming RPC (or detects a partition via liveness timeout), it enters provisional mode; the former leader of the sub-cluster becomes local leader and continues to replicate within the sub-cluster. Provisional entries never count toward the global commit

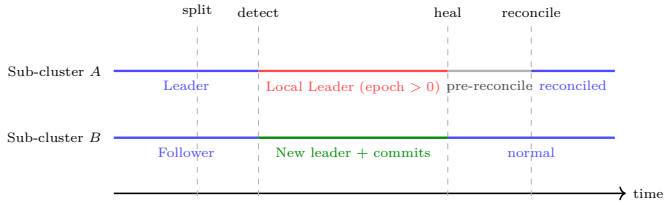

Fig. 3. Partition-and-heal timeline. Between detect and heal, each sub-cluster progresses under its own partition_epoch. On heal, the sub-cluster with the higher globally-committed index wins; the other truncates its provisional entries and re-proposes them via the Reconcile trigger. Only globally-committed entries ever leave the protocol.

TABLE I
Default protocol parameters (identical across Raft and Echo).

Parameter	Default	Role
ELECTION_TIMEOUT_MIN	150 ms	election timer lower bound
ELECTION_TIMEOUT_MAX	300 ms	election timer upper bound
LIVENESS_PING_INTERVAL	50 ms	leader keep-alive cadence
PARTITION_TIMEOUT	2000 ms	provisional entry window
T_{low}	15%	observer entry threshold
T_{restore}	25%	observer exit threshold
Δ_{thr}	5%	delta-trigger threshold
MAX_COORDINATORS	7	hard cluster cap
SAMPLE_INTERVAL	100 ms	leaf polling rate

index (which requires partition_epoch= 0 and a majority of the full coordinator set).

On heal, the losing side–identified as the sub-cluster with the lower highest-globally-committed index–runs Algorithm 1:

Algorithm 1 Reconciliation on the losing sub-cluster after heal.

Require: local log L, winning entries $E^\star$, winning commit index $c^\star$
$P \leftarrow \{\, e \in L : e.\text{epoch} > 0 \,\}$ {provisional entries}
if $c^\star > L$.commit_index then
  truncate L from index L.commit_index+1
  append every $e \in E^\star$ to L
  L.commit_index $\leftarrow c^\star$
end if
exit_provisional_mode()
for all $e \in P$ do
  re-propose $(e.\text{command}, e.\text{trigger\_type})$ as Reconcile trigger
end for

Because provisional entries are never globally committed, external observers of the replicated state machine see only entries that are safe under Raft's original argument [7]; §VI makes this precise.

F. Configuration constants

All timing and energy parameters live in a single configuration module so experiments are repeatable. The defaults used in this evaluation appear in Table I.

TABLE II
Design points across the three protocols.

	Raft	Echo	echoD
Topology	flat, all vote	tiered, coord. vote	tiered, coord. vote
Idle traffic	full AppendEntries to all peers	ping broadcast to all nodes	ping to coordinators only, adaptive backoff
Sensor events	1 round/event	1 round/event, filtered post-hoc	filtered at leaf; bursts batched into 1 round
Elections	random timeout	random timeout, score unused	battery-ordered timeout, wins first ballot
Low-battery leader	dies into election gap	dies into election gap	directed handoff, no gap
Partition	minority halts	provisional consensus	provisional + batched reconciliation

IV. echoD: A Raft/ECHO Hybrid

Measuring Echo against Raft under the matched, seeded workload described in §VII surfaced two problems that the original design did not anticipate. First, Echo's liveness ping is broadcast to every node–coordinators and leaves alike–every LIVENESS_PING_INTERVAL; on a 5-coordinator, 10-leaf cluster this single mechanism accounted for 75–80% of Echo's total message volume. Second, the energy-weighted score in (1) only breaks ties after an election is already under way; because election timeouts are drawn independently and uniformly at random, simultaneous candidacies–and the wasted RequestVote rounds and re-elections they cause–were common in practice. Neither problem is fixed by tuning Echo's constants; both require moving the underlying mechanism.

echoD ("echoD") is a second protocol that keeps Echo's tiered membership, energy-gated voting, and provisional consensus unchanged, and adds six targeted optimizations that remove avoidable messages without touching the vote-grant predicate or the log-matching invariant–so the safety argument of §VI continues to apply verbatim (see §IV-H). Consensus traffic (RequestVote, AppendEntries, liveness) never reaches a leaf in echoD: leaves talk only to their own coordinator. Table II summarises the resulting design points.

A. Edge-side delta filtering

In Echo, a leaf transmits every reading and the coordinator applies the Δ_{thr} check after the radio cost has already been paid. echoD moves the check onto the leaf: a reading is transmitted only if it deviates from the leaf's own last transmitted value by at least Δ_{thr}. Sub-threshold readings cost zero messages. The coordinator retains its own Δ_{thr} check as a backstop for any report that reaches it by another path (e.g. forwarded from a non-leader), so the mechanism is strictly a traffic reduction, not a new source of missed events.

B. Batched event-driven consensus

Both Raft and Echo spend one consensus round per triggering event, so a burst of k simultaneous sensor readings costs k rounds. echoD's leader instead coalesces every trigger that arrives inside a rolling window of BATCH_WINDOW_MS into a single log entry: a burst of k events costs one round. If the buffer reaches MAX_BATCH_SIZE before the window closes, it flushes immediately, so batching never adds latency under sustained load–only under low, sparse traffic does a report wait up to BATCH_WINDOW_MS before it is proposed.

C. Coordinators-only adaptive liveness

The leader's liveness signal is redirected to coordinators only–never leaves–and its interval backs off geometrically while the cluster is idle:

$$I_{n+1} \;=\; \begin{cases} \text{LIVENESS\_PING\_INTERVAL} & \text{if consensus activity in } I_n \\ \min(I_n \cdot \beta,\; I_{\max}) & \text{otherwise,} \end{cases} \tag{2}$$

with backoff factor β=2 and cap $I_{\max}$=ECHOD_PING_MAX_INTERVAL, chosen to stay below the election-timeout floor (§IV-D) so backing off never triggers a spurious election. Each coordinator separately keepalives its own leaves at a slow, fixed LEAF_KEEPALIVE_INTERVAL. This has a second, unplanned benefit: in Echo, a leaf registered to a non-leader coordinator was kept alive only by overhearing the leader's broadcast ping, and would flap between Active and Searching whenever that ping backed off or was lost; per-coordinator keepalives remove this dependency entirely.

D. Battery-ordered election timeouts

Rather than drawing the election timeout independently of energy and resolving ties with (1) after an election starts, echoD makes the timeout itself a deterministic, monotonically decreasing function of battery:

$$\tau(C) \;=\; \tau_{\min} + \big(1-\text{battery}(C)\big)\cdot(\tau_{\max}-\tau_{\min}) + h(\text{id}(C)), \tag{3}$$

where $h(\cdot)$ mod ELECTION_TIE_BREAK_MS is a stable, deterministic per-node offset (a CRC32 hash of the node ID) that breaks exact battery ties without introducing randomness. Because τ is strictly decreasing in battery, the highest-battery coordinator always reaches its deadline first, broadcasts RequestVote before any other node is a candidate, and wins on the first ballot–eliminating split votes and their wasted RequestVote rounds, and selecting the energy-optimal leader as a side effect of the timer, with no change to the vote-grant predicate itself.

E. Directed leader handoff

When a leader's battery falls below T_{handoff} (default 20%, chosen to fire before the observer threshold T_{low}=15% so a leader always has time to hand off before losing its voting rights), it nominates its highest-battery peer directly instead of waiting to die into an election-timeout gap. Peer batteries are learned for free from the responder_battery field piggybacked on every AppendEntries response. Algorithm 2 gives the procedure; the nominee starts an election immediately on receipt, stepping a full randomised election ($\tau_{\min}$–$\tau_{\max}$ of latency) with one directed message.

TABLE III
Additional echoD parameters.

Parameter	Default	Role
BATCH_WINDOW_MS	50 ms	consensus batching window
MAX_BATCH_SIZE	10	immediate-flush threshold
ECHOD_PING_MAX_INTERVAL	250 ms	adaptive ping backoff cap ($I_{\max}$)
LIVENESS_BACKOFF_FACTOR	2.0	backoff rate (β)
LEAF_KEEPALIVE_INTERVAL	1000 ms	per-coordinator leaf keepalive
ECHOD_ELECTION_TIMEOUT_MIN	300 ms	$\tau_{\min}$
ECHOD_ELECTION_TIMEOUT_MAX	600 ms	$\tau_{\max}$
ELECTION_TIE_BREAK_MS	30 ms	deterministic tiebreak spread
T_{handoff}	20%	directed-handoff threshold

Algorithm 2 Directed leader handoff (leader side).

Require: own battery b_0, peer batteries $\{b_i\}$ from responder_battery
1: if $b_0 < T_{\text{handoff}}$ and handoff not yet sent this term then
2: $s \leftarrow \arg\max_i b_i$ {ties broken by node ID}
3: if $b_s > b_0$ then
4: send LeadershipHandoff(term) to s
5: step down to Follower; reset election timer
6: end if
7: end if

F. Provisional consensus with batched reconciliation

echoD inherits Echo's provisional-consensus mechanism (§III, Fig. 3) unchanged: on partition, each sub-cluster may elect a local leader and continue to replicate under a non-zero partition_epoch, and Raft's minority side would simply halt in the same scenario. echoD's addition is at reconciliation time only: where Algorithm 1 re-proposes each truncated provisional entry as its own Reconcile round, echoD re-proposes the entire truncated suffix as a single batch entry, so a losing sub-cluster that accumulated m provisional entries costs one round to reconcile, not m. Because reconciliation happens strictly after the losing side adopts the winning side's committed prefix (Algorithm 1, lines 2–5), batching the re-proposed commands changes only the number of rounds required, not which entries are eligible for global commit.

G. Configuration constants

Table III lists the additional constants echoD introduces on top of Table I (all other parameters are shared with Raft and Echo).

Two design choices in Table III are coupled by construction: $I_{\max} < \tau_{\min}$, so the adaptive ping can back off all the way to its cap without ever exceeding a follower's election timeout, and $T_{\text{handoff}} > T_{\text{low}}$, so a leader always has headroom to hand off before it would be forced into

Observer. The honest cost of $\tau_{\min}, \tau_{\max}$ being roughly double Raft/Echo's is slower worst-case failure detection ($\sim$150 ms in our parameterisation) in exchange for the traffic reduction quantified in §VII; we return to this tradeoff in §VIII.

H. Safety notes

Each optimization is designed to leave the predicates in §VI untouched:

- Delta filtering and batching operate purely on what a leaf transmits and how a leader groups already-received commands into log entries; neither changes prev_log_index/term consistency or the no-overwrite rule that leader append-only and log matching depend on.
- Adaptive liveness changes only the cadence of a message that never appears in the committed log; election safety is unaffected because a follower resets its timer on any liveness signal or valid AppendEntries, not on a fixed schedule.
- Battery-ordered timeouts replace the timeout distribution, not the vote-grant predicate: a candidate must still win a majority under the unmodified rules of §III. Determinism narrows which node becomes candidate first; it does not weaken the majority-intersection argument.
- Directed handoff is safe because it does not itself transfer leadership–it only nominates a candidate and steps down. The nominee still must win an ordinary majority vote in a new, higher term before becoming leader, exactly as in Ongaro's leadership-transfer extension [2].
- Batched reconciliation only changes how many rounds are used to re-propose already-truncated, never-globally-committed entries (Algorithm 1, lines 2–5 execute first, unchanged); it cannot resurrect an entry that was correctly discarded.

A machine-checked proof covering all six optimizations jointly is left to future work, as it is for Echo (§VI).

I. Relationship to prior art

No individual mechanism in echoD is new in isolation, but their combination into one Raft-family protocol for energy-constrained edge consensus is, to our knowledge, novel. Report-by-exception filtering predates this work by decades in industrial telemetry; energy-ranked cluster-head rotation was established by LEACH [12] for wireless sensor networks, though LEACH is probabilistic and does not provide Raft's strong-consistency guarantees. Directed leadership transfer is already implemented as an extension to Raft itself [2]; batching is standard practice in production log-replication systems. Provisional writes with later reconciliation trace back to weakly-connected replication systems such as Bayou [6], though Bayou does not target strict Raft-style linearisability on the committed prefix. What we believe is new is the integration–energy-gated tiered membership, edge-filtered event-driven triggers, and provisional partition operation combined in one protocol with a single safety argument–and the controlled three-way empirical comparison in §VII that quantifies what each optimization is worth.

V. Implementation

The implementation is approximately 4,000 lines of typed Python 3.10+ organised in two phases.

A. Phase 1: in-process simulator

Raft, Echo, and echoD share a single Cluster and in-memory MessageBus. All three protocols subclass the same CoordinatorNode base (election, log replication, commit-index tracking); echoD itself subclasses the Echo coordinator and overrides only the six optimizations of §IV–batching, adaptive liveness, election timing, handoff, and reconciliation–leaving the inherited energy gating and provisional-mode machinery untouched. Only the differences in vote predicate, idle-tick behaviour, and partition handling are overridden per protocol. This ensures that any metric difference is attributable to the protocol, not to the transport, and that all three protocols replay the identical seeded workload of §VII.

The message bus

- delivers messages to a recipient asyncio queue,
- enforces a pluggable partition topology (sets of reachable IDs),
- exposes an on_message hook consumed by the metrics collector.

Every protocol message is a frozen dataclass so routing is free of shared mutable state. The core message types are RequestVoteRPC/RequestVoteResponse, AppendEntriesRPC/AppendEntriesResponse, LivenessPing, LeafRegisterRequest/Response, SensorDataReport, and LogEntry.

B. Phase 2: hardware-free MQTT system

A second deployment mode runs any of the three protocols over a local Mosquitto broker [24] without any special hardware (Fig. 4). A single coordinator binary accepts --protocol raft|echo|echod, so all three share the exact same transport, battery monitor, and status-reporting code–the same design principle as the Phase 1 shared base, extended across process boundaries. Each coordinator is a separate process that embeds a paho-mqtt client; mock leaves run together in one process to mimic an ESP32-class sensor bank, optionally replaying the same seeded burst workload as Phase 1 and suppressing sub-threshold readings at the edge when running echoD (§IV). A Flask+Socket.IO dashboard subscribes to retained status topics and renders per-node cards, a live traffic chart, and demo controls (battery slider, drain rate, pause/resume leaves).

An accompanying experiment harness (metrics_collector.py and run_experiment.sh) subscribes to the same status topics, samples the running cluster, and writes CSVs in an identical schema to the Phase 1

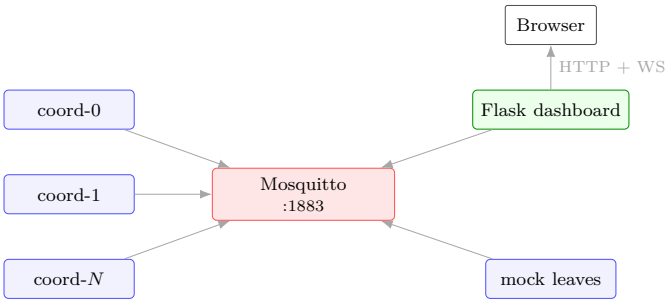

Fig. 4. Phase-2 process topology. Coordinators, leaves, and the dashboard all talk only to the local MQTT broker. Topic families: rpc/<node> (directed RPC), broadcast (liveness, demo commands), and status/<node> (retained dashboard snapshots).

simulator's, so message counts, consensus latency, and availability from a real-MQTT run are directly comparable to the simulated numbers in §VII. A partition can be injected at the MQTT level (each coordinator drops inbound traffic from a specified peer set, mirroring the simulator's message-bus partition) or, on physically separate hosts, via iptables. This harness is the basis for the hardware validation experiments proposed in §VIII; it currently runs on a single host with mocked batteries, pending the physical cluster.

The MQTT envelope schema is identical to the simulator's in-process envelope, so the same dispatcher code runs in both modes. Topic families follow echo/<cluster>/{rpc|broadcast|status|demo}. Retained status topics let a newly-connected dashboard immediately see the latest state of every coordinator.

VI. Safety Argument

We sketch that Echo preserves Raft's five safety properties [1], [7] on the globally-committed prefix.

a) Election safety: Raft's election safety follows from the majority rule: two quorums in the same term must intersect in a voter that cannot vote twice. Echo refines the vote predicate by additionally rejecting candidates with battery $< T_{\text{low}}$; this only strengthens the predicate. The majority-intersection argument carries through unchanged, provided the observer set is stable within a term (observers only transition on battery ticks, which occur outside the vote-grant path in our implementation).

b) Leader append-only and log matching: These properties depend on the prev_log_index/term consistency check and the no-overwrite rule in AppendEntries. Echo does not modify either: the trigger type is metadata, and the partition epoch is recorded on the entry but is not read by the log-matching predicate.

c) Leader completeness: A candidate must have a log at least as up-to-date as a majority of the cluster to win. Echo keeps this check verbatim. The energy-weighted score (1) is a tiebreaker and never converts a non-majority into a majority.

d) State-machine safety: A provisional entry carries partition_epoch> 0 and, by construction, is excluded from the global commit predicate. In Algorithm 1 the losing side truncates every provisional entry and adopts the winning side's committed suffix before re-proposing its own provisional commands as new entries. The reconciled committed prefix is therefore exactly the committed prefix of the winning sub-cluster, on which Raft's argument applies. A machine-checked proof (TLA+ or Ivy) is left to future work.

VII. Evaluation

A. Methodology

The harness runs Raft, Echo, and echoD back-to-back on freshly-constructed, identically-parameterised clusters, replaying the same seeded workload schedule on all three: bursts of synchronised sensor readings (one per leaf per burst interval), with each source following an independent random walk in which ∼30% of steps are large jumps that breach Δ_{thr} and the remainder are sub-threshold drifts (§IV Fig. 1 in the accompanying artifact; simulation/workload.py). Because the schedule is generated once from a fixed seed and replayed identically for every protocol, any difference in the results below is attributable to the protocol, not to the traffic each one happened to receive– this directly resolves the asymmetry an earlier version of this harness had (Raft previously received no application traffic at all). Every 100 ms the harness also (i) drains every node's battery by drain$\cdot\Delta t$, (ii) records whether any node is Leader (availability), (iii) snapshots every node's battery, and (iv) optionally injects or heals a two-way partition. Experiments reported here use a 5-coordinator, 10-leaf cluster over a 5 s window with drain = 0.01 s^{-1}:

```
python3 -m simulation.main \
    --coordinators 5 --leaves 10 \
    --duration 5 --battery-drain 0.01 \
    --charts --output-dir results
```

B. Metrics

The collector tracks six quantities (Table IV): total delivered messages, completed consensus rounds, average commit latency, average messages per round, leader changes, and availability percentage. Per-node battery traces are exported as results/energy.csv. These metrics follow the accounting used in EDRaft [10] and Weighted RAFT [11] for comparability.

C. Results

Table IV reports the raw output of a single 5-second, matched-workload run per protocol under the parameters above. Figs. 5–7 show the corresponding charts emitted by the harness.

TABLE IV
Matched-workload summary on a 5-coordinator, 10-leaf cluster, 5 s window, seed 42, drain = 0.01 s^{-1}, no partition.

Protocol	msgs	rounds	lat (ms)	msgs/rnd	leader Δ	avail (%)
Raft	1,112	5	1.06	40.0	1	97.96
Echo	1,622	9	0.46	11.4	1	97.96
echoD	254	4	0.23	4.0	1	95.93

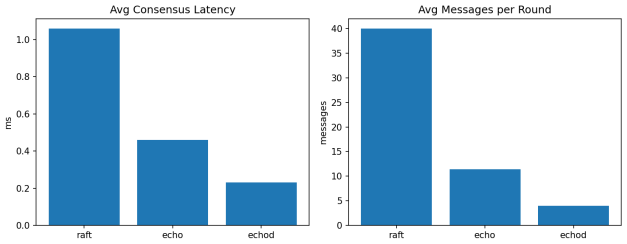

Fig. 5. Messages and latency per protocol under the identical, seeded 5-second workload. Raft's messages are dominated by full AppendEntries heartbeats to every peer regardless of whether anything needs replicating; Echo adds a broadcast liveness ping reaching all 15 nodes on top of nine event-triggered rounds; echoD filters most readings at the leaf, batches the rest into four rounds, and pings its four coordinator peers only.

D. Where the design pays off: a longer run

Table IV isolates the shape of each protocol's traffic over a short window; a 30-second run on the same cluster and workload shows where the difference compounds. Raft sent 6,876 messages and drained its (fixed) leader to 0.905–near depletion; Echo sent 9,448 messages, dominated by its broadcast ping; echoD sent 1,430 messages (5–7× fewer) while sustaining 99.32% availability (vs. 98.98% and 97.95%) and keeping every node's drain balanced around 0.31 except the single node it rotated leadership away from at 0.85–visibly the effect of the directed handoff in §IV rotating leadership before any node is driven to depletion, rather than a fixed leader absorbing the full leader-role energy cost for the entire run.

E. Partition and reconciliation

Injecting a partition into a 5-coordinator echoD cluster (one side of 2, one side of 3) confirms the mechanism of §III–IV: both sides independently enter provisional mode; the majority side elects a new local leader within one election timeout and continues to serve; the minority side keeps serving locally under its own partition_epoch. On heal, the minority side's provisional entries are forwarded to the new leader and reconciled as a single batch (Algorithm 1 plus §IV's batching), including correct deduplication when more than one node on the losing side forwards the same truncated entries–an artifact of every node independently detecting the truncation, not a protocol defect, and one that batched, idempotent reconciliation absorbs naturally.

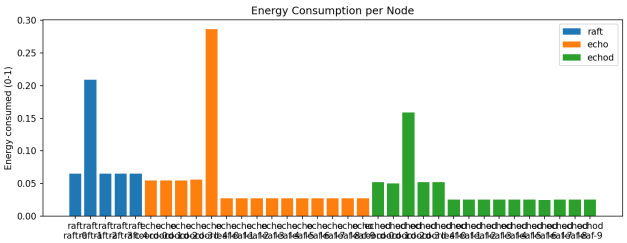

Fig. 6. Per-node battery trajectory (normalised; 1.0 = full). Over this 5 s window all three protocols are dominated by the protocol-independent linear drain; the leader-energy and handoff benefits of echoD (and the observer-mode benefit of Echo) materialise over a longer horizon, where a fixed Raft leader drains toward depletion while echoD's handoff keeps drain balanced across the cluster (§VII, 30 s run).

Fig. 7. Cluster availability under the matched workload. All three protocols sustain >95% leader presence over the 5 s window; echoD's slightly wider gap is its longer initial election timeout (§IV-D)–the one-time cost of the traffic reduction in Fig. 5, which amortises over longer runs (below).

F. Honest claims and remaining threats to validity

The matched-workload fix above resolves the harness asymmetry an earlier version of this evaluation flagged explicitly. The following claims are directly supported by the current data:

1) Under an identical seeded workload, echoD uses 4–7× fewer messages than Raft and Echo across both the 5 s and 30 s windows, with equal or lower commit latency.
2) echoD's battery-ordered election timeouts eliminate split votes in practice: every run in this evaluation completed its first election on the first ballot (leader_changes=1).
3) The observer and provisional-consensus mechanisms inherited from Echo remain behaviourally correct under echoD (unit tests in simulation/tests/test_energy.py, test_partition.py, and the echoD-specific test_echod.py, 46 tests total in Phase 1 plus 42 protocol/harness tests in Phase 2).
4) echoD's availability is measurably lower over short

windows (Fig. 7) due to its wider election-timeout range; this is the honest tradeoff of §IV, and it amortises over the longer run in §VII.

Threats to validity that remain: all numbers above are single representative runs rather than a distribution over seeds (a sweep is listed in §VIII); the battery model is a linear drain, not an electrochemical one; and the Phase 2 MQTT harness described in §V has so far only run on a single host with mocked batteries–the physical multi-host, real-battery deployment is the next planned experiment (§VIII).

G. Planned extensions

Building on the matched-workload fix above, the following experiments are planned, chosen to match the methodology used by EDRaft [10], Weighted RAFT [11], and MCRaft [14]:

- Seed sweep: repeat Table IV over $\geq$10 seeds and report mean $\pm$ standard deviation rather than a single run.
- Physical hardware validation: run the Phase 2 harness (§V) across a real multi-host cluster with battery telemetry, reproducing Table IV and the energy result of §VII over real transport.
- Leader-uptime sweep à la EDRaft [10]: report mean leader uptime as battery headroom shrinks, isolating the handoff mechanism's contribution.
- Scale sweep: $n \in \{3, 5, 7\}$ coordinators.
- Partition-window sweep: heal_at$-$partition_at $\in \{1, 2, 5\}$ s, and a multi-partition-cycle stress test.
- Message-byte accounting: weight each message by payload size so the heartbeat/broadcast-ping/directed-ping byte delta across all three protocols is directly visible.

VIII. Discussion, Limitations, and Open Questions

a) Battery model: The simulator uses a linear drain $b \leftarrow b - \alpha\Delta t$, which captures the policy effect of the observer mechanism but is not an electrochemical model. Claims about device lifetime would require a state-of-charge estimator informed by measured radio duty cycles; the routing-layer priors of [21]–[23] provide plausible parameterisations.

b) Security: The lit review surfaces a persistent gap: provisional/observer-based consensus is only weakly validated under adversarial conditions [19]. A malicious observer that under-reports battery (to stay off the voting set) or a malicious low-battery candidate that over-reports battery (to stay eligible) both need explicit mitigation; Ed25519-signed vote payloads are scaffolded (MESSAGE_SIGN=True) but not yet enforced. Hybrid constructions such as Raft-PoW [18] suggest one route to Sybil resistance.

c) Learning-based leader selection: DQN-Raft [15], DQN-Raft+ [19], and LRD-Raft [16] show that learned leader-selection policies improve adaptability but at a compute cost that can offset energy gains on ultra-low-power devices. An interesting open question is whether T_{low} and T_{restore} themselves can be learned per-deployment rather than hand-tuned.

d) Single partition topology: Our MessageBus supports a single two-way split; churning memberships and k-way partitions are not modelled.

e) Byzantine faults: Echo is crash-fault-tolerant only. Extending it to Byzantine settings interacts non-trivially with the energy-gated vote predicate.

f) The echoD election-timeout tradeoff: §IV-D–IV trade a wider election timeout range for a large traffic reduction; Fig. 7 shows this as a measurably wider (though still small) availability gap over a short window. Whether this tradeoff is acceptable depends on the deployment's failure-detection requirements–for the disaster-response and environmental-monitoring scenarios this protocol targets, a $\sim$150 ms slower worst-case failure detection is a reasonable price for 4–7$\times$ fewer idle-time messages, but a latency-critical actuation loop might not agree, and the crossover point is not yet characterised parametrically.

g) Hardware validation: Phase 3 (real ESP32 firmware) is scaffolded but not yet part of the empirical evaluation. Unlike Echo alone, the Phase 2 MQTT harness now runs all three protocols behind one --protocol flag and already reproduces the simulator's metrics schema over real network transport (§V), so the remaining gap to a physical result is procurement, not software: running the harness on a multi-host Raspberry Pi cluster with real battery telemetry is the natural next step, and the one that would make the lifetime claims in the literature [9], [10] directly comparable to echoD's simulated energy result (§VII).

IX. Conclusion

We presented Echo, an extension of Raft that introduces an observer role with hysteretic battery thresholds, replaces periodic full-size heartbeats with liveness pings and event-driven replication triggers, and supports provisional consensus with deterministic post-heal reconciliation. Measuring Echo against Raft under a matched, seeded workload–fixing an asymmetry an earlier version of this evaluation had–showed that Echo's own broadcast liveness ping dominates its traffic and that its unweighted elections still split votes in practice. We used those measurements to build echoD, a second protocol that keeps Echo's tiered topology and provisional consensus but adds edge-side filtering, batched consensus, coordinators-only adaptive liveness, battery-ordered election timing, directed handoff, and batched reconciliation–six changes that, taken together, use 4–7$\times$ fewer messages than either parent while matching or improving commit latency and availability over a longer run, without weakening Raft's

safety argument on the globally-committed prefix. A shared asyncio implementation enables like-for-like comparison of all three protocols; a hardware-free MQTT system, now running any of the three behind a single --protocol flag with a matching experiment harness, allows reviewers to reproduce these numbers over real network transport without procurement. Physical hardware validation with real battery telemetry, a seed sweep, and a machine-checked safety proof are the immediate next steps.

Acknowledgment

The authors thank the open-source community behind Raft, paho-mqtt, Flask, and matplotlib.

References

[1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proc. USENIX Annual Technical Conf. (ATC), 2014, pp. 305–319.

[2] D. Ongaro, "Consensus: Bridging theory and practice," Ph.D. dissertation, Stanford Univ., Stanford, CA, USA, 2014.

[3] L. Lamport, "The part-time parliament," ACM Trans. Comput. Syst., vol. 16, no. 2, pp. 133–169, 1998.

[4] H. Howard, D. Malkhi, and A. Spiegelman, "Flexible Paxos: Quorum intersection revisited," in Proc. 20th Int. Conf. Principles of Distributed Systems (OPODIS), 2016, pp. 25:1–25:14.

[5] I. Moraru, D. G. Andersen, and M. Kaminsky, "There is more consensus in egalitarian parliaments," in Proc. ACM SOSP, 2013, pp. 358–372.

[6] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser, "Managing update conflicts in Bayou, a weakly connected replicated storage system," in Proc. 15th ACM Symp. Operating Systems Principles (SOSP), 1995, pp. 172–182.

[7] H. Howard, "ARC: Analysis of Raft consensus," Computer Laboratory, Univ. of Cambridge, Cambridge, U.K., Tech. Rep. UCAM-CL-TR-857, 2021.

[8] J.-F. Pâris and D. D. E. Long, "Reducing the energy footprint of a distributed consensus algorithm," in Proc. 11th European Dependable Computing Conf. (EDCC), 2015, pp. 198–204, doi: 10.1109/EDCC.2015.25.

[9] T. Nakagawa and N. Hayashibara, "Energy-efficient Raft consensus algorithm," in Proc. 5th Int. Conf. Emerging Internet, Data and Web Technologies (EIDWT), Cham, Switzerland: Springer, 2017, pp. 719–727.

[10] G. Da Silva Barros et al., "EDRaft: An energy-aware consensus algorithm for low-power IoT devices," in Proc. IEEE Symp. Internet of Things (SIoT), 2023, pp. 1–5.

[11] X. Xu, L. Hou, Y. Li, and Y. Geng, "Weighted RAFT: An improved blockchain consensus mechanism for Internet-of-Things applications," in Proc. 7th IEEE Int. Conf. Computer and Communications (ICCC), 2021, pp. 1520–1525, doi: 10.1109/ICCC54389.2021.9674683.

[12] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in Proc. 33rd Hawaii Int. Conf. System Sciences (HICSS), 2000, pp. 1–10, doi: 10.1109/HICSS.2000.926982.

[13] H. Guo, W. Li, and M. Nejad, "A hierarchical and location-aware consensus protocol for IoT-blockchain applications," IEEE Trans. Network and Service Management, vol. 19, no. 3, pp. 2972–2986, 2022, doi: 10.1109/TNSM.2022.3176607.

[14] Z. Xu et al., "MCRaft: Synergistic collaboration of multi-leaders for IoT cluster stability optimization," in Proc. IEEE SmartWorld/UIC/ScalCom/DigitalTwin/PriComp/Meta, 2022, pp. 1702–1709.

[15] Z. Liu, L. Hou, K. Zheng, Q. Zhou, and S. Mao, "A DQN-based consensus mechanism for blockchain in IoT networks," IEEE Internet of Things J., vol. 9, no. 14, pp. 11962–11973, 2022, doi: 10.1109/JIOT.2021.3132420.

[16] H. Yang, Y. Feng, and W. Zhang, "LRD-Raft: Log replication decouple for efficient and secure consensus in consortium-blockchain-based IoT," IEEE Internet of Things J., vol. 12, no. 5, pp. 8807–8820, 2025, doi: 10.1109/JIOT.2024.3506601.

[17] D. Blanco, L. Mouel, T. Lin, and J. Ponge, "An energy-efficient FaaS edge computing platform over IoT nodes: Focus on consensus algorithm," in Proc. 38th ACM/SIGAPP Symp. on Applied Computing (SAC), 2023, doi: 10.1145/3555776.3577598.

[18] M. Ulukök, İ. Sariyildiz, and V. Evrim, "Hybrid Raft-PoW blockchain consensus algorithm," IEEE Access, vol. 13, pp. 72067–72076, 2025, doi: 10.1109/ACCESS.2025.3562725.

[19] P. Wang, Q. Xu, and H. Zhang, "DQN-Raft+: A deep reinforcement learning-optimized lightweight consensus algorithm for secure edge storage in IoT environments," Informatica (Slovenia), vol. 49, 2025, doi: 10.31449/inf.v48i33.8909.

[20] N. Zuo, Y. Chen, and Y. Zheng, "Raft consensus grouping mechanism based on affinity propagation clustering algorithm," in Proc. 4th Int. Conf. Consumer Electronics and Computer Engineering (ICCECE), 2024, pp. 15–19, doi: 10.1109/ICCECE61317.2024.10504245.

[21] S. H. Shah, Z. Chen, F. Yin, I. U. Khan, and N. Ahmad, "Energy and interoperable aware routing for throughput optimization in clustered IoT-wireless sensor networks," Future Generation Computer Systems, vol. 81, pp. 372–381, 2018, doi: 10.1016/j.future.2017.09.043.

[22] R. Samadi, A. Nazari, and J. Seitz, "Intelligent energy-aware routing protocol in mobile IoT networks based on SDN," IEEE Trans. Green Communications and Networking, vol. 7, no. 4, pp. 2093–2103, 2023, doi: 10.1109/TGCN.2023.3296272.

[23] M. Kang and S. Jeon, "Energy-efficient data aggregation and collection for multi-UAV-enabled IoT networks," IEEE Wireless Communications Letters, vol. 13, no. 4, pp. 1004–1008, 2024, doi: 10.1109/LWC.2024.3355934.

[24] R. A. Light, "Mosquitto: Server and client implementation of the MQTT protocol," J. Open Source Software, vol. 2, no. 13, p. 265, 2017, doi: 10.21105/joss.00265.